```
// ================== WEATHER GATE v2.5 - WORKING ==================
#define MQTT_MAX_PACKET_SIZE 1024

#include <ETH.h>
#include <WiFi.h>
#include <Wire.h>
#include <RTClib.h>
#include <SPI.h>
#include <LittleFS.h>
#include <ArduinoJson.h>
#include <WebServer.h>
#include <PubSubClient.h>
#include <Preferences.h>
#include <HTTPClient.h>
#include <HTTPUpdate.h>
#include <esp_task_wdt.h>
#include <time.h>
#include <math.h>
#include <rtl_433_ESP.h>
#include <RadioLib.h>

extern CC1101 radio;

// ================== КОНФИГУРАЦИЯ ==================
SET_LOOP_TASK_STACK_SIZE(20 * 1024);
#define CURRENT_VERSION "2.5"
#define FW_NAME "weather_gate"

// ================== ЧАСОВОЙ ПОЯС ==================
#define TIMEZONE_OFFSET 7

// ================== ПИНЫ ==================
#define I2C_SDA 32
#define I2C_SCL 33
#define LED_RF_RX 25
#define LED_MQTT_TX 26
#define PIN_BAT_ADC 36
#define PIN_POWER_SENSE 39
#define CC1101_GDO0 4

// ================== НАСТРОЙКИ ==================
#define POINTS_HISTORY 288
#define HISTORY_INTERVAL 300000
#define SENSOR_INTERVAL 60000
```

```
#define MQTT_PUBLISH_INTERVAL 60000
#define MQTT_RECONNECT_INTERVAL 10000
#define SMART_SAVE_NORMAL 300000
#define SMART_SAVE_LOW 60000
#define HA_PORT 8123
#define HA_OTA_PATH "/local/ota/weather_gate/"
#define MQTT_TOPIC_WEATHER_DEFAULT "weather_gate/data"

// ==================== ПАРАМЕТРЫ СКАНЕРА ====================
const int SCAN_STEPS = 21;
const float SCAN_START_FREQ = 914.80;
const float SCAN_STEP = 0.02;
const unsigned long SCAN_STEP_TIME = 65000;

// ==================== УСРЕДНЕНИЕ ВЕТРА ====================
const int WIND_AVG_SAMPLES = 10;
const int WIND_SPEED_SAMPLES = 10;

// ==================== ZAMBRETTI ====================
const int PRESSURE_HISTORY_SIZE = 12;
float pressure_history[PRESSURE_HISTORY_SIZE];
int pressure_history_idx = 0;
bool pressure_history_full = false;
float pressure_trend_3h = 0.0;
String zambretti_forecast = "Ожидание данных...";
String pressure_trend_desc = "Стабильно";
unsigned long lastZambrettiUpdate = 0;

// ==================== LOG МАКРОС ====================
#define LOG(tag, msg, ...) Serial.printf("[%08lu] [%s] " msg "\n", millis(), tag, ##__VA_ARGS__)

// ==================== СТРУКТУРЫ ====================
struct RollingHistory {
float temp[POINTS_HISTORY];
float pressure[POINTS_HISTORY];
float wind[POINTS_HISTORY];
int head = 0;
} history;

struct UpdateInfo {
String version;
String fw_url;
String fs_url;
String fw_md5;
```

```
String fs_md5;
String changelog;
};

// ====================== ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ ======================
bool rtc_found = false;
RTC_DS3231 rtc;

float temperature = 0, humidity = 0, pressure = 1013.25;
float wind_speed_raw = 0, wind_gust_raw = 0;
int wind_direction_raw = 0;
float rain = 0;
unsigned long last_packet_time = 0;

float temp_min = 0, temp_max = 0;
float pressure_min = 1013.25, pressure_max = 1013.25;
float wind_max = 0;
float rain_max = 0;

String outdoor_battery = "OK";
int sensor_id = -1;
float last_rssi = -120.0;

// Усреднение ветра
float wind_dir_history_sin[WIND_AVG_SAMPLES];
float wind_dir_history_cos[WIND_AVG_SAMPLES];
int wind_sample_idx = 0;
bool wind_history_filled = false;
int wind_dir_avg = 0;

float wind_speed_history[WIND_SPEED_SAMPLES];
int wind_speed_idx = 0;
bool wind_speed_filled = false;
float wind_avg_smooth = 0;
float wind_gust_max = 0;

// Настройки устройства
float battery_voltage = 0;
float battery_coeff = 2.1;
float current_freq = 915.00;
bool verbose_mode = false;

String hostname = "weather_gate";
String mqtt_ip = "";
```

```
String mqtt_user = "";
String mqtt_password = "";
String mqtt_topic_weather = MQTT_TOPIC_WEATHER_DEFAULT;

uint32_t radio_errors = 0, radio_success = 0;
uint32_t lastSaveTime = 0, lastHistoryTime = 0;
uint32_t lastMQTTReconnect = 0, lastMQTTPublish = 0;

// Переменные сканера
bool scan_active = false;
float scan_results[SCAN_STEPS];
int scan_current_step = 0;
float scan_best_freq = 915.00;
float scan_best_rssi = -150.0;
float scan_step_max_rssi = -150.0;
unsigned long scan_step_start = 0;
float scan_instant_rssi = -120.0;

WebServer server(80);
Preferences pref;
WiFiClient ethClient;
PubSubClient mqtt(ethClient);
rtl_433_ESP rf;
char msgBuffer[512];

// ==================== ПРОТОТИПЫ ====================
void loadSettings();
void saveAll();
void saveCounters();
void sendDiscovery(const char* name, const char* unit, const char* dev_cla);
void registerAllSensors();
void publishMQTTData();
void readSensors();
void mqttCallback(char* topic, byte* payload, unsigned int length);
void reconnectMQTT();
void addHistory(float t, float p, float w);
void smartSave();
bool getUpdateInfo(UpdateInfo &info);
int compareVersions(String newVer, String curVer);
void performOTA();
void printResetReason();
String formatDateTime(DateTime dt);
float parseSafeFloat(String val);
bool handleFileRead(String path);
```

```
String getWindDirectionText(int degrees);
void updateWindAverage(int degrees);
void updateWindSpeedAverage(float current_speed, float current_gust);
void updateAutoScanner();
void updatePressureHistory(float currentPressure);
float calculatePressureTrend();
void calculateZambrettiForecast();

// ==================== ФУНКЦИИ ====================
String formatDateTime(DateTime dt) {
time_t utc = dt.unixtime();
time_t local = utc + TIMEZONE_OFFSET * 3600;
struct tm *timeinfo = localtime(&local);
char buffer[20];
strftime(buffer, sizeof(buffer), "%d.%m.%Y %H:%M:%S", timeinfo);
return String(buffer);
}

float parseSafeFloat(String val) {
val.replace(',', '.');
val.trim();
return val.toFloat();
}

String getBatteryStatus() {
if (battery_voltage < 0.5) return "НЕТ";
if (battery_voltage < 1.0) return "НЕТ";
if (battery_voltage < 3.4 && !digitalRead(PIN_POWER_SENSE)) return "РАЗРЯЖЕН";
return "ОК";
}

int getBatteryState() {
if (battery_voltage < 0.5) return 0;
if (battery_voltage < 1.0) return 0;
if (battery_voltage < 3.4 && !digitalRead(PIN_POWER_SENSE)) return 1;
return 2;
}

void addHistory(float t, float p, float w) {
history.temp[history.head] = t;
history.pressure[history.head] = p;
history.wind[history.head] = w;
history.head = (history.head + 1) % POINTS_HISTORY;
}
```

```
void printResetReason() {
esp_reset_reason_t reason = esp_reset_reason();
LOG("SYS", "=== RESET DIAGNOSTIC ===");
switch(reason) {
case ESP_RST_POWERON: LOG("SYS", "Reset: POWER_ON"); break;
case ESP_RST_EXT: LOG("SYS", "Reset: EXTERNAL"); break;
case ESP_RST_SW: LOG("SYS", "Reset: SOFTWARE"); break;
case ESP_RST_PANIC: LOG("SYS", "Reset: PANIC"); break;
case ESP_RST_INT_WDT: LOG("SYS", "Reset: INT WDT"); break;
case ESP_RST_TASK_WDT: LOG("SYS", "Reset: TASK WDT"); break;
case ESP_RST_BROWNOUT: LOG("SYS", "Reset: BROWNOUT"); break;
default: LOG("SYS", "Reset: UNKNOWN (%d)", reason); break;
}
LOG("SYS", "Free heap: %u bytes", ESP.getFreeHeap());
LOG("SYS", "==========================");
}

String getWindDirectionText(int degrees) {
const char* directions[] = {"С", "СВ", "В", "ЮВ", "Ю", "ЮЗ", "З", "СЗ"};
int idx = (degrees + 22) / 45 % 8;
return String(directions[idx]);
}

void updateWindAverage(int degrees) {
float rad = degrees * M_PI / 180.0;
wind_dir_history_sin[wind_sample_idx] = sin(rad);
wind_dir_history_cos[wind_sample_idx] = cos(rad);
wind_sample_idx++;
if (wind_sample_idx >= WIND_AVG_SAMPLES) {
wind_sample_idx = 0;
wind_history_filled = true;
}
int count = wind_history_filled ? WIND_AVG_SAMPLES : wind_sample_idx;
float avg_sin = 0, avg_cos = 0;
for (int i = 0; i < count; i++) {
avg_sin += wind_dir_history_sin[i];
avg_cos += wind_dir_history_cos[i];
}
avg_sin /= count;
avg_cos /= count;
float avg_rad = atan2(avg_sin, avg_cos);
wind_dir_avg = (int)(avg_rad * 180.0 / M_PI);
if (wind_dir_avg < 0) wind_dir_avg += 360;
```

```
}

void updateWindSpeedAverage(float current_speed, float current_gust) {
wind_speed_history[wind_speed_idx] = current_speed;
wind_speed_idx++;
if (wind_speed_idx >= WIND_SPEED_SAMPLES) {
wind_speed_idx = 0;
wind_speed_filled = true;
}
float sum = 0;
int count = wind_speed_filled ? WIND_SPEED_SAMPLES : wind_speed_idx;
for (int i = 0; i < count; i++) sum += wind_speed_history[i];
wind_avg_smooth = sum / count;
wind_gust_max = current_gust;
}

// ==================== ZAMBRETTI ====================
void updatePressureHistory(float currentPressure) {
pressure_history[pressure_history_idx] = currentPressure;
pressure_history_idx = (pressure_history_idx + 1) % PRESSURE_HISTORY_SIZE;
if (pressure_history_idx == 0) pressure_history_full = true;
}

float calculatePressureTrend() {
if (!pressure_history_full) return 0.0;
int firstIdx = pressure_history_idx;
int lastIdx = (pressure_history_idx - 1 + PRESSURE_HISTORY_SIZE) % PRESSURE_HISTORY_SIZE;
float firstPressure = pressure_history[firstIdx];
float lastPressure = pressure_history[lastIdx];
return (lastPressure - firstPressure);
}

void calculateZambrettiForecast() {
float trend = calculatePressureTrend();
if (trend > 1.5) pressure_trend_desc = "Резкий подъём ↑↑";
else if (trend > 0.5) pressure_trend_desc = "Подъём ↑";
else if (trend < -1.5) pressure_trend_desc = "Резкий спад ↓↓";
else if (trend < -0.5) pressure_trend_desc = "Спад ↓";
else pressure_trend_desc = "Стабильно →";
float currentPressure = pressure;
if (currentPressure > 1025.0 && trend > 0.0) zambretti_forecast = "Отличная, ясная погода ☀️";
else if (currentPressure > 1025.0) zambretti_forecast = "Ясная погода ☀️";
else if (currentPressure > 1020.0 && trend < -0.5) zambretti_forecast = "Переменная облачность ⛅";
else if (currentPressure > 1020.0) zambretti_forecast = "Хорошая погода ☀️";
```

```
else if (currentPressure > 1015.0 && trend > 0.0) zambretti_forecast = "Улучшение погоды ☀️";
else if (currentPressure > 1015.0) zambretti_forecast = "Облачно с прояснениями ⛅";
else if (currentPressure > 1010.0 && trend < -0.5) zambretti_forecast = "Вероятны осадки 🌧️";
else if (currentPressure > 1010.0) zambretti_forecast = "Облачно ⛅";
else if (currentPressure > 1005.0 && trend < -0.5) zambretti_forecast = "Дождливая погода 🌧️";
else if (currentPressure > 1005.0) zambretti_forecast = "Пасмурно ☁️";
else if (currentPressure > 1000.0) zambretti_forecast = "Дожди 🌧️";
else zambretti_forecast = "Штормовое предупреждение ⛈️";
LOG("ZAMBRETTI", "Прогноз: %s (Давление: %.1f hPa, Тренд: %.2f)",
zambretti_forecast.c_str(), currentPressure, trend);
}

// ==================== СКАНЕР ====================
void updateAutoScanner() {
if (!scan_active) return;

scan_instant_rssi = radio.getRSSI();

if (scan_step_start == 0) {
scan_step_start = millis();
float freq = SCAN_START_FREQ + (scan_current_step * SCAN_STEP);
radio.setFrequency(freq);
radio.setRxBandwidth(325.0);
radio.setFrequencyDeviation(47.6);
radio.setBitRate(17.241);
radio.startReceive();
delay(20);
scan_step_max_rssi = radio.getRSSI();
LOG("SCAN", "Step %d start: freq=%.3f MHz, init RSSI=%.1f",
scan_current_step, freq, scan_step_max_rssi);
}

if (scan_instant_rssi > scan_step_max_rssi)
scan_step_max_rssi = scan_instant_rssi;

if (millis() - scan_step_start > SCAN_STEP_TIME) {
scan_results[scan_current_step] = scan_step_max_rssi;
LOG("SCAN", "Step %d complete: freq=%.3f MHz, max RSSI=%.1f dBm",
scan_current_step, SCAN_START_FREQ + (scan_current_step * SCAN_STEP), scan_step_max_rssi);

if (scan_step_max_rssi > scan_best_rssi) {
scan_best_rssi = scan_step_max_rssi;
scan_best_freq = SCAN_START_FREQ + (scan_current_step * SCAN_STEP);
LOG("SCAN", ">>> New best: %.3f MHz (RSSI: %.1f dBm) <<<", scan_best_freq, scan_best_rssi);
```

```
}

radio.standby();
scan_current_step++;
scan_step_start = 0;

if (scan_current_step >= SCAN_STEPS) {
scan_active = false;
rf.initReceiver(CC1101_GDO0, current_freq);
rf.enableReceiver();

if (scan_best_rssi > -95.0) {
current_freq = scan_best_freq;
radio.setFrequency(current_freq);
saveAll();
LOG("SCAN", "✅ OPTIMAL FREQUENCY FOUND: %.3f MHz (RSSI: %.1f dBm)", current_freq, scan_best_rssi);
} else {
LOG("SCAN", "⚠️ No strong signal found, keeping freq: %.3f MHz", current_freq);
}
} else {
delay(10);
}
}
}

// ==================== MQTT ====================
void mqttCallback(char* topic, byte* payload, unsigned int length) {}

void sendDiscovery(const char* name, const char* unit, const char* dev_cla) {
if (!mqtt.connected()) return;
String deviceId = hostname;
if (deviceId.length() == 0) deviceId = "weather_gate";
String topic = "homeassistant/sensor/" + deviceId + "/" + name + "/config";
DynamicJsonDocument doc(1024);
doc["name"] = deviceId + " " + name;
doc["state_topic"] = ("smart/" + deviceId + "/state").c_str();
doc["value_template"] = ("{{ value_json." + String(name) + " }}").c_str();
doc["unique_id"] = (deviceId + "_" + name).c_str();
doc["unit_of_measurement"] = unit;
if (dev_cla) doc["device_class"] = dev_cla;
JsonObject device = doc.createNestedObject("device");
device["identifiers"] = deviceId;
device["name"] = "Weather Gate";
device["sw_version"] = CURRENT_VERSION;
```

```
device["model"] = "WS1080 Gateway";
device["manufacturer"] = "DIY";
String out;
serializeJson(doc, out);
mqtt.publish(topic.c_str(), out.c_str(), true);
}

void registerAllSensors() {
delay(100);
sendDiscovery("temperature", "°C", "temperature");
delay(50);
sendDiscovery("humidity", "%", "humidity");
delay(50);
sendDiscovery("pressure", "hPa", "pressure");
delay(50);
sendDiscovery("wind_speed", "m/s", "wind_speed");
delay(50);
sendDiscovery("wind_gust", "m/s", "wind_speed");
delay(50);
sendDiscovery("wind_direction", "°", NULL);
delay(50);
sendDiscovery("rain", "mm", "precipitation");
delay(50);
sendDiscovery("outdoor_battery", NULL, "battery");
}

void publishMQTTData() {
if (!mqtt.connected()) return;
DynamicJsonDocument doc(1024);
doc["temperature"] = temperature;
doc["humidity"] = humidity;
doc["pressure"] = pressure;
doc["wind_speed"] = wind_avg_smooth;
doc["wind_gust"] = wind_gust_max;
doc["wind_direction"] = wind_dir_avg;
doc["rain"] = rain;
doc["outdoor_battery"] = outdoor_battery;
doc["sensor_id"] = sensor_id;
doc["battery"] = battery_voltage;
doc["battery_status"] = getBatteryStatus();
doc["power_source"] = digitalRead(PIN_POWER_SENSE) ? "BAT" : "AC";
doc["uptime"] = millis() / 1000;
doc["radio_errors"] = radio_errors;
doc["radio_success"] = radio_success;
```

```
doc["current_freq"] = current_freq;
doc["rssi"] = last_rssi;
if (rtc_found) doc["datetime"] = formatDateTime(rtc.now());
String out;
serializeJson(doc, out);
String stateTopic = "smart/" + hostname + "/state";
mqtt.publish(stateTopic.c_str(), out.c_str(), false);
}

void reconnectMQTT() {
if (mqtt_ip.length() < 7) return;
String id = hostname;
if (id.length() == 0) id = "weather_gate";
String clientId = "WeatherGate_" + id + "_" + String(ESP.getEfuseMac(), HEX);
if (mqtt.connect(clientId.c_str(), mqtt_user.c_str(), mqtt_password.c_str())) {
LOG("MQTT", "Connected to broker");
registerAllSensors();
} else {
LOG("MQTT", "Failed, rc=%d", mqtt.state());
}
}

// ==================== RTL_433 CALLBACK ====================
void rtl_433_Callback(char* message) {
digitalWrite(LED_RF_RX, HIGH);
last_rssi = radio.getRSSI();
JsonDocument doc;
if (!deserializeJson(doc, message)) {
radio_success++;
last_packet_time = millis();
if (doc.containsKey("id")) sensor_id = doc["id"];
if (doc.containsKey("temperature_C")) temperature = doc["temperature_C"];
else if (doc.containsKey("temp_C")) temperature = doc["temp_C"];
if (doc.containsKey("humidity")) humidity = doc["humidity"];
if (doc.containsKey("wind_avg_m_s")) wind_speed_raw = doc["wind_avg_m_s"];
if (doc.containsKey("wind_gust_m_s")) wind_gust_raw = doc["wind_gust_m_s"];
if (doc.containsKey("wind_dir_deg")) wind_direction_raw = doc["wind_dir_deg"];
if (doc.containsKey("rain_mm")) rain = doc["rain_mm"];
else if (doc.containsKey("rain")) rain = doc["rain"];
if (doc.containsKey("wind_avg_m_s") && doc.containsKey("wind_gust_m_s")) {
updateWindSpeedAverage(wind_speed_raw, wind_gust_raw);
}
if (doc.containsKey("wind_dir_deg")) {
updateWindAverage(wind_direction_raw);
```

```
}
if (doc.containsKey("battery_ok")) {
outdoor_battery = (doc["battery_ok"] == 1) ? "OK" : "LOW";
}
if (doc.containsKey("pressure_hPa")) pressure = doc["pressure_hPa"];
if (temperature < temp_min || temp_min == 0) temp_min = temperature;
if (temperature > temp_max) temp_max = temperature;
if (pressure < pressure_min) pressure_min = pressure;
if (pressure > pressure_max) pressure_max = pressure;
if (wind_avg_smooth > wind_max) wind_max = wind_avg_smooth;
if (rain > rain_max) rain_max = rain;
LOG("WS1080", "ID=%d T=%.1f°C H=%.0f%% W=%.1fm/s D=%d° Rain=%.1fmm Batt=%s RSSI=%.1f",
sensor_id, temperature, humidity, wind_avg_smooth, wind_dir_avg, rain, outdoor_battery.c_str(), last_rssi);
if (mqtt.connected()) {
digitalWrite(LED_MQTT_TX, HIGH);
publishMQTTData();
delay(20);
digitalWrite(LED_MQTT_TX, LOW);
}
} else {
radio_errors++;
LOG("RTL", "Parse error");
}
digitalWrite(LED_RF_RX, LOW);
}

void readSensors() {
int raw = 0;
for (int i = 0; i < 5; i++) raw += analogRead(PIN_BAT_ADC);
battery_voltage = (raw / 5.0) * (3.3 / 4095.0) * battery_coeff;
}

void smartSave() {
int state = getBatteryState();
unsigned long saveInterval = SMART_SAVE_NORMAL;
if (state == 1) saveInterval = SMART_SAVE_LOW;
if (millis() - lastSaveTime >= saveInterval) {
saveCounters();
lastSaveTime = millis();
LOG("SYS", "Smart save");
}
}

void loadSettings() {
```

```
pref.begin("weather", true);
hostname = pref.getString("hostname", "weather_gate");
if (hostname.length() == 0) hostname = "weather_gate";
mqtt_ip = pref.getString("mq_ip", "");
mqtt_user = pref.getString("mq_u", "");
mqtt_password = pref.getString("mq_p", "");
mqtt_topic_weather = pref.getString("mqtt_topic_weather", MQTT_TOPIC_WEATHER_DEFAULT);
current_freq = pref.getFloat("freq", 915.0);
battery_coeff = pref.getFloat("bat_k", 2.1);
radio_errors = pref.getUInt("radio_errors", 0);
radio_success = pref.getUInt("radio_success", 0);
temperature = pref.getFloat("temp", 0);
humidity = pref.getFloat("hum", 0);
pressure = pref.getFloat("pres", 1013.25);
temp_min = pref.getFloat("temp_min", 0);
temp_max = pref.getFloat("temp_max", 0);
pressure_min = pref.getFloat("pres_min", 1013.25);
pressure_max = pref.getFloat("pres_max", 1013.25);
wind_max = pref.getFloat("wind_max", 0);
rain = pref.getFloat("rain", 0);
rain_max = pref.getFloat("rain_max", 0);
sensor_id = pref.getInt("sensor_id", -1);
pref.end();
LOG("SYS", "Settings loaded, freq=%.2f MHz", current_freq);
}

void saveAll() {
pref.begin("weather", false);
pref.putString("hostname", hostname);
pref.putString("mq_ip", mqtt_ip);
pref.putString("mq_u", mqtt_user);
pref.putString("mq_p", mqtt_password);
pref.putString("mqtt_topic_weather", mqtt_topic_weather);
pref.putFloat("freq", current_freq);
pref.putFloat("bat_k", battery_coeff);
pref.putUInt("radio_errors", radio_errors);
pref.putUInt("radio_success", radio_success);
pref.putFloat("temp", temperature);
pref.putFloat("hum", humidity);
pref.putFloat("pres", pressure);
pref.putFloat("rain", rain);
pref.putFloat("rain_max", rain_max);
pref.putFloat("temp_min", temp_min);
pref.putFloat("temp_max", temp_max);
```

```
pref.putFloat("pressure_min", pressure_min);
pref.putFloat("pressure_max", pressure_max);
pref.putFloat("wind_max", wind_max);
pref.putInt("sensor_id", sensor_id);
pref.end();
LOG("SYS", "All settings saved");
}

void saveCounters() {
pref.begin("weather", false);
pref.putUInt("radio_errors", radio_errors);
pref.putUInt("radio_success", radio_success);
pref.putFloat("temp", temperature);
pref.putFloat("hum", humidity);
pref.putFloat("pres", pressure);
pref.putFloat("rain", rain);
pref.end();
}

// ==================== OTA ====================
bool getUpdateInfo(UpdateInfo &info) {
if (mqtt_ip.length() < 7) return false;
String url = "http://" + mqtt_ip + ":" + String(HA_PORT) + HA_OTA_PATH + "version.json";
HTTPClient http;
http.setTimeout(5000);
if (!http.begin(url)) return false;
int httpCode = http.GET();
if (httpCode != HTTP_CODE_OK) {
http.end();
return false;
}
String payload = http.getString();
http.end();
StaticJsonDocument<768> doc;
DeserializationError error = deserializeJson(doc, payload);
if (error) return false;
info.version = doc["version"] | "";
info.fw_md5 = doc["fw_md5"] | "";
info.fs_md5 = doc["fs_md5"] | "0";
info.changelog = doc["changelog"] | "";
String baseUrl = "http://" + mqtt_ip + ":" + String(HA_PORT) + HA_OTA_PATH;
info.fw_url = baseUrl + "firmware.bin";
info.fs_url = baseUrl + "littelfs.bin";
return info.version.length() > 0;
```

```
}

int compareVersions(String newVer, String curVer) {
if (newVer == curVer) return 0;
int newMajor = 0, newMinor = 0, newPatch = 0;
int curMajor = 0, curMinor = 0, curPatch = 0;
sscanf(newVer.c_str(), "%d.%d.%d", &newMajor, &newMinor, &newPatch);
sscanf(curVer.c_str(), "%d.%d.%d", &curMajor, &curMinor, &curPatch);
if (newMajor > curMajor) return 1;
if (newMajor < curMajor) return -1;
if (newMinor > curMinor) return 1;
if (newMinor < curMinor) return -1;
if (newPatch > curPatch) return 1;
if (newPatch < curPatch) return -1;
return 0;
}

void performOTA() {
LOG("OTA", "=== OTA START ===");
UpdateInfo info;
if (!getUpdateInfo(info)) return;
if (info.version == CURRENT_VERSION) return;
LOG("OTA", "Updating to %s", info.version.c_str());
mqtt.disconnect();
delay(100);
WiFiClient otaClient;
otaClient.setTimeout(12000);
if (info.fs_md5.length() > 0 && info.fs_md5 != "0") {
LOG("OTA", "Updating LittleFS...");
httpUpdate.setMD5sum(info.fs_md5);
httpUpdate.updateFs(otaClient, info.fs_url);
delay(500);
}
LOG("OTA", "Updating firmware...");
httpUpdate.setMD5sum(info.fw_md5);
httpUpdate.rebootOnUpdate(true);
httpUpdate.update(otaClient, info.fw_url);
}

bool handleFileRead(String path) {
if (path.endsWith("/")) path += "index.html";
String contentType = "text/html";
if (path.endsWith(".css")) contentType = "text/css";
else if (path.endsWith(".js")) contentType = "application/javascript";
```

```
else if (path.endsWith(".json")) contentType = "application/json";
else if (path.endsWith(".ico")) contentType = "image/x-icon";
String gzPath = path + ".gz";
if (LittleFS.exists(gzPath)) {
File file = LittleFS.open(gzPath, "r");
server.setContentLength(file.size());
server.sendHeader("Content-Encoding", "gzip");
server.send(200, contentType, "");
uint8_t buffer[512];
size_t bytesRead;
while ((bytesRead = file.read(buffer, sizeof(buffer))) > 0) {
server.client().write(buffer, bytesRead);
}
file.close();
return true;
}
if (LittleFS.exists(path)) {
File file = LittleFS.open(path, "r");
server.streamFile(file, contentType);
file.close();
return true;
}
return false;
}

// ===================== SETUP =====================
void setup() {
Serial.begin(115200);
delay(500);
printResetReason();

pinMode(LED_RF_RX, OUTPUT);
pinMode(LED_MQTT_TX, OUTPUT);
pinMode(PIN_POWER_SENSE, INPUT);
digitalWrite(LED_RF_RX, LOW);
digitalWrite(LED_MQTT_TX, LOW);
analogSetAttenuation(ADC_11db);

for (int i = 0; i < WIND_AVG_SAMPLES; i++) {
wind_dir_history_sin[i] = 0;
wind_dir_history_cos[i] = 0;
wind_speed_history[i] = 0;
}
for (int i = 0; i < SCAN_STEPS; i++) scan_results[i] = -120.0;
```

```
Wire.begin(I2C_SDA, I2C_SCL);
if (rtc.begin()) {
rtc_found = true;
LOG("RTC", "DS3231 found");
}

if (!LittleFS.begin(true)) LOG("FS", "LittleFS mount failed");
else LOG("FS", "LittleFS mounted");

WiFi.mode(WIFI_OFF);
btStop();
ETH.begin(ETH_PHY_LAN8720, 1, 23, 18, 16, ETH_CLOCK_GPIO17_OUT);
if (hostname.length() > 0) ETH.setHostname(hostname.c_str());
delay(500);
unsigned long startWait = millis();
while (ETH.localIP().toString() == "0.0.0.0" && millis() - startWait < 10000) delay(100);
LOG("ETH", "IP: %s", ETH.localIP().toString().c_str());

loadSettings();

LOG("CC1101", "Initializing...");
rf.setCallback(rtl_433_Callback, msgBuffer, 512);
rf.initReceiver(CC1101_GDO0, current_freq);
rf.enableReceiver();
LOG("CC1101", "Receiver ready at %.2f MHz", current_freq);

if (mqtt_ip.length() > 5) {
mqtt.setServer(mqtt_ip.c_str(), 1883);
mqtt.setCallback(mqttCallback);
LOG("MQTT", "Server set to %s", mqtt_ip.c_str());
}

// Инициализация истории давления (ВАЖНО!)
for (int i = 0; i < PRESSURE_HISTORY_SIZE; i++) pressure_history[i] = pressure;
pressure_history_idx = 0;
pressure_history_full = false;

// ==================== WEB-СЕРВЕР ====================
server.on("/api/data", HTTP_GET, []() {
DynamicJsonDocument doc(2048);
doc["temperature"] = temperature;
doc["humidity"] = humidity;
doc["pressure"] = pressure;
```

```
doc["wind_speed"] = wind_avg_smooth;
doc["wind_gust"] = wind_gust_max;
doc["wind_direction"] = wind_dir_avg;
doc["wind_dir_txt"] = getWindDirectionText(wind_dir_avg);
doc["rain"] = rain;
doc["rain_max"] = rain_max;
doc["temp_min"] = temp_min;
doc["temp_max"] = temp_max;
doc["pressure_min"] = pressure_min;
doc["pressure_max"] = pressure_max;
doc["wind_max"] = wind_max;
doc["outdoor_battery"] = outdoor_battery;
doc["sensor_id"] = sensor_id;
doc["battery"] = battery_voltage;
doc["battery_status"] = getBatteryStatus();
doc["power_source"] = digitalRead(PIN_POWER_SENSE) ? "BAT" : "AC";
doc["mqtt_status"] = mqtt.connected() ? "OK" : "Offline";
doc["hostname"] = hostname;
doc["version"] = CURRENT_VERSION;
doc["uptime"] = millis() / 1000;
doc["radio_errors"] = radio_errors;
doc["radio_success"] = radio_success;
doc["current_freq"] = current_freq;
doc["battery_coeff"] = battery_coeff;
doc["mqtt_ip"] = mqtt_ip;
doc["mqtt_user"] = mqtt_user;
doc["mqtt_topic_weather"] = mqtt_topic_weather;
doc["forecast"] = zambretti_forecast;
doc["pressure_trend"] = pressure_trend_desc;
doc["rssi"] = last_rssi;
if (rtc_found) doc["datetime"] = formatDateTime(rtc.now());
String buf;
serializeJson(doc, buf);
server.send(200, "application/json", buf);
});

server.on("/api/history", HTTP_GET, []() {
DynamicJsonDocument doc(4096);
JsonArray tempHist = doc.createNestedArray("temp_history");
JsonArray pressureHist = doc.createNestedArray("pressure_history");
JsonArray windHist = doc.createNestedArray("wind_history");
for (int i = 0; i < POINTS_HISTORY; i++) {
int idx = (history.head + i) % POINTS_HISTORY;
tempHist.add(history.temp[idx]);
```

```
pressureHist.add(history.pressure[idx]);
windHist.add(history.wind[idx]);
}
String buf;
serializeJson(doc, buf);
server.send(200, "application/json", buf);
});

server.on("/api/sync_time", HTTP_POST, []() {
if (server.hasArg("t") && rtc_found) {
rtc.adjust(DateTime(server.arg("t").toInt()));
server.send(200, "text/plain", "OK");
} else {
server.send(400, "text/plain", "Failed");
}
});

server.on("/api/set_freq", HTTP_POST, []() {
if (server.hasArg("f")) {
float new_freq = server.arg("f").toFloat();
if (new_freq >= 860.0 && new_freq <= 930.0) {
current_freq = new_freq;
pref.begin("weather", false);
pref.putFloat("freq", current_freq);
pref.end();
rf.disableReceiver();
delay(50);
rf.initReceiver(CC1101_GDO0, current_freq);
rf.enableReceiver();
LOG("RF", "Frequency changed to %.2f MHz", current_freq);
server.send(200, "text/plain", "OK");
} else {
server.send(400, "text/plain", "Invalid frequency");
}
}
});

server.on("/api/set_verbose", HTTP_POST, []() {
if (server.hasArg("v")) {
verbose_mode = (server.arg("v") == "1");
rf.rtlVerbose = verbose_mode ? 1 : 0;
LOG("SYS", "Verbose mode: %s", verbose_mode ? "ON" : "OFF");
server.send(200, "text/plain", "OK");
}
```

```
});

server.on("/api/reset_errors", HTTP_POST, []() {
radio_errors = 0;
radio_success = 0;
saveCounters();
server.send(200, "text/plain", "OK");
});

server.on("/api/check_update", HTTP_GET, []() {
DynamicJsonDocument doc(512);
UpdateInfo info;
if (getUpdateInfo(info)) {
bool hasUpdate = (info.version != CURRENT_VERSION);
doc["has_update"] = hasUpdate;
doc["current_version"] = CURRENT_VERSION;
doc["new_version"] = info.version;
doc["changelog"] = info.changelog;
doc["fs_md5"] = info.fs_md5;
} else {
doc["has_update"] = false;
doc["error"] = "Failed to fetch update info";
}
String buf;
serializeJson(doc, buf);
server.send(200, "application/json", buf);
});

server.on("/api/do_update", HTTP_POST, []() {
static unsigned long lastOTA = 0;
if (millis() - lastOTA < 60000) {
server.send(429, "text/plain", "Too frequent requests");
return;
}
lastOTA = millis();
server.send(200, "text/plain", "Update started...");
delay(500);
performOTA();
});

server.on("/api/start_scan", HTTP_GET, []() {
if (!scan_active) {
rf.disableReceiver();
scan_active = true;
```

```
scan_current_step = 0;
scan_best_rssi = -150.0;
scan_step_start = 0;
for (int i = 0; i < SCAN_STEPS; i++) scan_results[i] = -120.0;
LOG("API", "Spectrum scan started, receiver disabled");
server.send(200, "text/plain", "OK");
} else {
server.send(409, "text/plain", "Scan already active");
}
});

server.on("/api/scan_data", HTTP_GET, []() {
DynamicJsonDocument doc(1024);
doc["active"] = scan_active;
doc["step"] = scan_current_step;
doc["instant"] = scan_instant_rssi;
doc["rssi_current"] = radio.getRSSI();
JsonArray data = doc.createNestedArray("rssi");
for (int i = 0; i < SCAN_STEPS; i++) data.add(scan_results[i]);
String buf;
serializeJson(doc, buf);
server.send(200, "application/json", buf);
});

server.on("/api/config", HTTP_POST, []() {
String body = server.arg("plain");
DynamicJsonDocument doc(2048);
if (!deserializeJson(doc, body)) {
if (doc.containsKey("hostname")) hostname = doc["hostname"].as<String>();
if (doc.containsKey("mq_ip")) mqtt_ip = doc["mq_ip"].as<String>();
if (doc.containsKey("mq_u")) mqtt_user = doc["mq_u"].as<String>();
if (doc.containsKey("mq_p")) mqtt_password = doc["mq_p"].as<String>();
if (doc.containsKey("mqtt_topic_weather")) mqtt_topic_weather = doc["mqtt_topic_weather"].as<String>();
if (doc.containsKey("bat_k")) battery_coeff = parseSafeFloat(doc["bat_k"].as<String>());
ETH.setHostname(hostname.c_str());
mqtt.setServer(mqtt_ip.c_str(), 1883);
saveAll();
server.send(200, "OK");
delay(500);
ESP.restart();
} else {
server.send(400, "text/plain", "JSON Error");
}
});
```

```
server.on("/", HTTP_GET, []() {
if (!handleFileRead("/")) server.send(404, "text/plain", "index.html not found");
});

server.onNotFound([]() {
if (!handleFileRead(server.uri())) server.send(404, "text/plain", "Not Found");
});

server.begin();
LOG("HTTP", "Server started");

for (int i = 0; i < POINTS_HISTORY; i++) {
history.temp[i] = NAN;
history.pressure[i] = NAN;
history.wind[i] = NAN;
}
lastHistoryTime = millis();
lastSaveTime = millis();
lastZambrettiUpdate = millis();

LOG("SYS", "Setup complete");
LOG("SYS", "Waiting for WS1080 packets at %.2f MHz", current_freq);
}

// ===================== LOOP =====================
void loop() {
server.handleClient();
yield();

static uint32_t lastSensors = 0;

if (millis() - lastSensors >= SENSOR_INTERVAL) {
lastSensors = millis();
readSensors();
}

if (scan_active) {
updateAutoScanner();
} else {
static uint32_t lastRfTask = 0;
if (millis() - lastRfTask >= 10) {
lastRfTask = millis();
rf.loop();
```

```
}
smartSave();
if (millis() - lastHistoryTime >= HISTORY_INTERVAL) {
lastHistoryTime = millis();
addHistory(temperature, pressure, wind_avg_smooth);
}
if (!mqtt.connected() && mqtt_ip.length() > 5 && millis() - lastMQTTReconnect >= MQTT_RECONNECT_INTERVAL) {
lastMQTTReconnect = millis();
reconnectMQTT();
}
mqtt.loop();
}

// Обновление прогноза Zambretti каждые 15 минут
if (millis() - lastZambrettiUpdate > 900000 && last_packet_time > 0) {
lastZambrettiUpdate = millis();
updatePressureHistory(pressure);
calculateZambrettiForecast();
}

delay(1);
}
```